Lecture 9 — Hard DP: SOS DP, Broken-Profile DP, the Aliens Trick, Slope Trick

> World Finals Level · Prerequisites: Year weeks 26–27 (bitmask, interval, digit DP), regional-05 (DP patterns), nationals-03 (CHT, Li Chao, divide & conquer DP, Knuth), worldfinals-03 (LP duality) · Estimated time: 2 × 90 min lecture + 3 h lab

1. Overview

Top-level DP problems are rarely about finding *a* recurrence — the recurrence is often obvious. They are about making an exponential or quadratic DP fit the limits by exploiting structure. This lecture covers four such structures:

1. **Sum over subsets (SOS) DP** — the subset lattice is a hypercube, and sums over sub-/supersets are multi-dimensional prefix sums: $O(n\,2^n)$ instead of $O(3^n)$. Ranked by popcount, the same idea yields **subset convolution** in $O(n^2\,2^n)$.
2. **Broken-profile DP** — grid DPs where the state is the frontier between processed and unprocessed cells, advanced one *cell* at a time: O(nm 2^m).
3. **The Aliens trick** (WQS binary search, Lagrangian relaxation) — eliminates the "exactly k" dimension of a DP whose optimum is concave in k.
4. **Slope trick** — when every DP layer is a convex piecewise-linear function, store only its breakpoints in two heaps.

All code in §6 was validated against brute force: SOS transforms and subset convolution against direct enumeration for $n \le 10$; domino tilings against backtracking for all boards up to 6×6 (and 12 988 816 for 8×8); Aliens against an $O(nk)$ DP for every k on 3000 random arrays; slope trick against an explicit function on a grid under random sequences of all operations.

2. SOS DP and subset transforms

2.1 Zeta and Möbius transforms on the subset lattice

Given f defined on subsets of an n-element set (bitmasks), the **zeta transform** is $F(S) = \Sigma_{T \subseteq S} f(T)$. Enumerating submasks of every mask costs $\Sigma_S 2^{|S|} = 3^n$, which is $3.5 \cdot 10^9$ for $n = 20$.

View masks as points of $\{0,1\}^n$. Then $T \subseteq S$ means $T \le S$ coordinate-wise, and F is an n-dimensional prefix sum. Just as 2D prefix sums can be computed one axis at a time, define $F_i(S) = \Sigma\, f(T)$ over $T \subseteq S$ that agree with S on bits $i, i+1, \ldots, n-1$. Then $F_0 = f$, $F_n = F$, and $F_{i+1}(S) = F_i(S) + [\text{bit } i \in S] \cdot F_i(S \setminus \{i\})$. In code: for each bit i (outer loop), for each mask S containing i, `f[S] += f[S ^ (1<<i)]`. The inner order of masks is irrelevant because step i only combines masks differing in bit i — but the **bit loop must be outermost**.

The inverse (**Möbius transform**) undoes the steps with subtraction; it is inclusion–exclusion in disguise: $f(S) = \Sigma_{T \subseteq S} (-1)^{|S \setminus T|} F(T)$. Superset sums are the same with the condition flipped.

The zeta step works for any commutative monoid, not just addition: max, min, OR, "the two largest indices" (CF 1208F), "any witness" (CF 165E).

2.2 Convolutions

- **OR convolution** $h(S) = \Sigma_{A \cup B = S} f(A)\, g(B)$: zeta both, multiply pointwise, Möbius back. Proof: $\zeta h(S) = \Sigma_{A \cup B \subseteq S} f(A) g(B) = \zeta f(S) \cdot \zeta g(S)$.
- **AND convolution**: the same with superset transforms.
- **XOR convolution**: Walsh–Hadamard transform (Lecture 2).
- **Subset convolution** $h(S) = \Sigma_{A \subseteq S} f(A)\, g(S \setminus A)$: the pairs are disjoint with union S, i.e. $A \cup B = S$ and $|A| + |B| = |S|$. Split f and g by popcount into $n + 1$ "ranked" arrays f_k, g_k, zeta each, compute $H_k = \Sigma_j \zeta f_j \cdot \zeta g_{k-j}$ pointwise, Möbius each H_k, and read $h(S)$ from $H_{|S|}(S)$. The ranks forbid overlap: if $A \cup B = S$ with $|A| + |B| = |S|$ then $A \cap B = \varnothing$. Time $O(n^2\, 2^n)$.

Subset convolution is the engine for set-partition DPs: "count ways to partition the vertex set into blocks with property P" is the exponential of a set power series, computable by n subset convolutions or by a ranked exp in $O(n^2\, 2^n)$.

2.3 Typical patterns

- **Counting subsets with AND = 0** (CF 449D): let $c(S)$ = number of elements that are supersets of S (superset zeta of the frequency array). The number of non-empty subsets whose AND contains S is $2^{c(S)} - 1$. By inclusion–exclusion, the number with AND exactly 0 is $\Sigma_S (-1)^{|S|} (2^{c(S)} - 1)$.
- **Compatible pairs** (CF 165E): for each a, find any b with $a \,\&\, b = 0$, i.e. $b \subseteq \sim a$. Store one witness per mask and propagate with a subset zeta that keeps any non-empty witness.
- **Max over submasks with two indices** (CF 1208F): keep the two largest indices among supersets of S; then greedily build the answer bit by bit from the highest.

3. Broken-profile DP

3.1 The frontier

Consider tiling an n × m grid with dominoes. Processing row by row with a 2^m mask of "cells of the next row already covered" requires enumerating all ways to fill a row — up to 2^m · 2^m transitions per row, too slow for m = 10 with n = 1000 (that is 10^9). Instead, process **one cell at a time** in row-major order. When we are at cell (i, j), the frontier consists of m cells: (i+1, 0..j−1) and (i, j..m−1). Bit k of the profile tells whether the frontier cell in column k is already covered.

At cell (i, j):

- if bit j is set, the cell was covered by a vertical domino from above: clear bit j (it now refers to (i+1, j), which is uncovered) and move on;
- otherwise place a vertical domino (set bit j, meaning (i+1, j) becomes covered) if i + 1 < n;
- or a horizontal domino (set bit j+1, meaning (i, j+1) is covered) if j + 1 < m and bit j+1 is clear.

Each cell does O(2^m) work, total O(nm 2^m) time and O(2^m) memory. For CSES 2181 (n ≤ 10, m ≤ 1000) the mask runs over the short side.

3.2 Variations

- **Obstacles**: treat blocked cells as already covered (they must not be covered again).
- **Optimisation**: replace addition by min/max over transitions with weights.
- **Larger pieces**: L-trominoes and 2 × 2 squares reach two rows ahead or two columns sideways; either enlarge the profile (two bits per column) or use a ternary profile.
- **Connectivity (plug DP)**: counting Hamiltonian cycles or simple paths on grids needs to know which frontier "plugs" are connected to each other. Encode the frontier as a bracket sequence (each path end is "(" or ")"), store states in a hash map, and handle the merge cases. This is the hardest variant and appears at ICPC as "count closed tours on a grid with obstacles".
- **Huge n with tiny m**: all rows are identical, so the row-to-row transition is a fixed 2^m × 2^m matrix T and the answer is (T^n)[0][0]; with m ≤ 5 matrix power is trivial. Alternatively compute the first few hundred terms with the cell DP and run Berlekamp–Massey (Lecture 2).

4. The Aliens trick

4.1 Setting

Many problems ask for the best value using **exactly k** (or at most k) objects: k segments, k groups, k facilities. A direct DP carries k as a dimension, e.g. $O(n^2k)$ or $O(nk)$. Let $f(k)$ be the optimum with exactly k objects (maximisation; for minimisation flip signs). Suppose f is **concave**: $f(k+1) - f(k)$ is non-increasing.

Penalise each object by a real λ and solve the unconstrained problem

$g(\lambda) = \max$ over all solutions (value $- \lambda \cdot$ #objects) $= \max_k (f(k) - \lambda k)$.

This is one DP without the k dimension. Geometrically, $g(\lambda)$ is the intercept of the supporting line with slope λ to the point set $\{(k, f(k))\}$; the line touches the graph exactly at the k's with $f(k) - f(k-1) \geq \lambda \geq f(k+1) - f(k)$. Because f is concave, every k is touched for some λ, and as λ grows the touching k decreases. Binary search on λ until the optimal count is k, then $f(k) = g(\lambda) + \lambda k$. This is Lagrangian duality (Lecture 3) for a one-dimensional constraint; concavity is precisely what makes the duality gap zero.

4.2 Ties and why consistency matters

If several consecutive k lie on one line (f is linear there), they are simultaneously optimal for that λ, and no λ makes k the *unique* optimum. The robust recipe:

1. In the penalised DP, break ties among equal values by preferring the **minimum** count — consistently, in every transition.
2. Binary search the **smallest** integer λ with $\text{cnt_min}(\lambda) \leq k$.
3. At that λ, k is among the optimal counts ($\text{cnt_min}(\lambda) \leq k \leq \text{cnt_max}(\lambda)$), so $f(k) = g(\lambda) + \lambda \cdot k$. Use k, **not** the returned count.

Why step 3 holds: with integer data all slopes are integers. Let $\lambda^* = f(k+1) - f(k)$ (when $k = n$ there is no $k+1$; the search then ends at the lower bound $-B$, where n is optimal). For $\lambda = \lambda^*$, both k and $k+1$ are optimal and $\text{cnt_min} \leq k$; for $\lambda = \lambda^* - 1$, the slope $f(k+1) - f(k) > \lambda$ so every optimal count is $\geq k+1$. So the smallest λ with $\text{cnt_min}(\lambda) \leq k$ is λ^*, at which k is optimal.

The search range must contain all slopes. For "k disjoint subarrays", one extra subarray can change the total by up to $\Sigma|a_i|$, so $\lambda \in [-\Sigma|a_i| - 1, \Sigma|a_i| + 1]$. (Our first test used $\pm 3 \cdot 10^9$ for values up to 10^9 and failed: a single slope can exceed $\max|a_i|$.)

4.3 Establishing concavity

The trick silently gives wrong answers if f is not concave, so you need an argument:

- **Flow/matroid structure.** If f(k) is the optimum of k augmentations in a min-cost flow, the successive shortest path costs are monotone, so f is concave. "Exactly k disjoint subarrays with maximum sum" is such a flow (source → positions → sink with unit capacities on segment starts).
- **Monge/quadrangle inequality.** For partition DPs dp_k(i) = min_j dp_{k−1}(j) + w(j, i) where w satisfies the quadrangle inequality, the optimum is convex in k. IOI 2016 *Aliens* (cover points with k squares along the diagonal) is of this type.
- **Exchange argument.** Show that from optimal solutions for k − 1 and k + 1 one can construct two solutions for k whose values sum to at least f(k−1) + f(k+1).
- **Empirical check.** Before the contest-time investment, compute f(k) for small random inputs with the slow DP and verify the second differences. Ten lines of code, and it has saved many teams.

4.4 Combining with other optimisations

After the k dimension is gone, the inner DP may still be quadratic; then apply convex hull trick or Li Chao trees (nationals-03). IOI 2016 *Aliens* is Aliens + CHT, O(n log n log C). CF 739E *Gosha is hunting* has two resources: apply Aliens to one and run an O(n · b) DP for the other, or apply it to both with nested binary searches. CF 1279F *New Year and Handle Change* uses Aliens over the number of length-l operations with an O(n) inner DP.

5. Slope trick

5.1 Representation

Let f: $\mathbb{Z} \to \mathbb{Z}$ be convex and piecewise linear with integer breakpoints and integer slopes. Write it as

$$f(x) = \text{minf} + \Sigma_{l \in L} \max(0, l - x) + \Sigma_{r \in R} \max(0, x - r),$$

where L and R are multisets and max(L) ≤ min(R). Each l ∈ L is a point where the slope increases by one on the left side, each r ∈ R one on the right side, and f attains its minimum minf on [max L, min R]. Store L in a max-heap and R in a min-heap, each with a lazy additive offset.

5.2 Operations

- **Add max(0, x − a).** If a ≥ max L, it simply joins R. Otherwise the new minimum shifts: the cost increases by max L − a, a enters L, and the old max L moves to R. The uniform code: `minf += max(0, topL - a); pushL(a); pushR(popL());`.
- **Add max(0, a − x).** Symmetric.
- **Add |x − a|** = both.

- **Prefix minimum** g(x) = min_{y ≤ x} f(y): the right side becomes flat — clear R.
- **Suffix minimum**: clear L.
- **Sliding window minimum** g(x) = min_{x−b ≤ y ≤ x−a} f(y) with a ≤ b: the left part shifts right by a and the right part by b — just add a to L's offset and b to R's offset.
- **Evaluate** f(x): minf plus the two sums — O(size), done once at the end if needed.

All operations are O(log n). Adding a general convex function with k breakpoints costs O(k log n). Two functions can be added by merging heaps (small-to-large gives O(n log² n) overall), which is what tree problems need.

5.3 Worked example: CF 713C

Make a strictly increasing with minimum $\Sigma|a_i - b_i|$. Substituting $c_i = a_i - i$ reduces to non-decreasing b. Let dp_i(x) = minimum cost for the first i elements with b_i = x. Then dp_i(x) = $|x - c_i|$ + min_{y ≤ x} dp_{i−1}(y): a prefix minimum followed by adding an absolute value. Answer: minf at the end. The well-known "one heap" solution (push c_i twice, pop once, add the difference) is exactly this code with the R heap discarded because each step clears it anyway.

5.4 Recognising slope-trick problems

- The DP state is a single integer coordinate (position, value, time) with a large range.
- Transitions are: add a convex cost depending on the coordinate, then take a min over a window or a half-line.
- Typical statements: "move a point along a line with bounded speed and penalties", "make a sequence monotone/with bounded differences at minimum L1 cost", "adjust edge lengths in a tree so all leaves are at equal depth" (APIO 2016 *Fireworks*), "points move to positions with unit cost per step" (CF 1534G after a 45° rotation).

6. Reference implementations

6.1 SOS DP — CF 449D *Jzzhu and Numbers*

Count non-empty subsets (by index) whose bitwise AND is 0, modulo $10^9 + 7$. n ≤ 10^6, $a_i < 2^{20}$.

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll MOD = 1'000'000'007;

int main() {
    const int B = 20, N = 1 << B;
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n;
    cin >> n;
```

```
    vector<int> c(N, 0);
    for (int i = 0; i < n; i++) { int a; cin >> a; c[a]++; }
    for (int i = 0; i < B; i++)                      // superset sums: c[S] = #elements
⊇ S
        for (int S = 0; S < N; S++)
            if (!(S >> i & 1)) c[S] += c[S | (1 << i)];
    vector<ll> p2(n + 1, 1);
    for (int i = 1; i <= n; i++) p2[i] = p2[i - 1] * 2 % MOD;
    ll ans = 0;
    for (int S = 0; S < N; S++) {
        ll term = (p2[c[S]] - 1 + MOD) % MOD;          // subsets whose AND ⊇ S
        ans = (__builtin_popcount(S) & 1) ? (ans - term + MOD) % MOD : (ans + term) %
MOD;
    }
    printf("%lld\n", ans);
}
```

6.2 Subset transforms and subset convolution

```
void subsetZeta(vector<ll>& f, int n) {
    for (int i = 0; i < n; i++) for (int S = 0; S < (1 << n); S++) if (S >> i & 1) f[S]
+= f[S ^ (1 << i)];
}
void subsetMobius(vector<ll>& f, int n) {
    for (int i = 0; i < n; i++) for (int S = 0; S < (1 << n); S++) if (S >> i & 1) f[S] -
= f[S ^ (1 << i)];
}
vector<ll> subsetConv(const vector<ll>& f, const vector<ll>& g, int n) {   // O(n^2 2^n)
    int N = 1 << n;
    vector<vector<ll>> F(n + 1, vector<ll>(N, 0)), G = F, H = F;
    for (int S = 0; S < N; S++) F[__builtin_popcount(S)][S] = f[S],
G[__builtin_popcount(S)][S] = g[S];
    for (int k = 0; k <= n; k++) subsetZeta(F[k], n), subsetZeta(G[k], n);
    for (int k = 0; k <= n; k++)
        for (int j = 0; j <= k; j++)
            for (int S = 0; S < N; S++) H[k][S] += F[j][S] * G[k - j][S];
    for (int k = 0; k <= n; k++) subsetMobius(H[k], n);
    vector<ll> h(N);
    for (int S = 0; S < N; S++) h[S] = H[__builtin_popcount(S)][S];
    return h;
}
```

For a modulus, reduce after each step. At $n = 20$ the ranked product is about $n^2/2 \cdot 2^n \approx 2 \cdot 10^8$ multiply-adds; use 32-bit unsigned modular arithmetic and keep the mask loop innermost (contiguous memory) to stay around a second.

6.3 Broken profile — CSES 2181 *Counting Tilings*

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
typedef long long ll;
const ll MOD = 1'000'000'007;

ll dominoTilings(int n, int m) {      // O(n m 2^min(n,m))
    if (m > n) swap(n, m);            // mask over the short side
    vector<ll> dp(1 << m, 0), nd(1 << m);
    dp[0] = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            // bit k < j: cell (i+1, k) covered; bit k >= j: cell (i, k) covered
            fill(nd.begin(), nd.end(), 0);
            for (int mask = 0; mask < (1 << m); mask++) {
                if (!dp[mask]) continue;
                ll v = dp[mask];
                if (mask >> j & 1) { nd[mask ^ (1 << j)] = (nd[mask ^ (1 << j)] + v) %
MOD; continue; }
                if (i + 1 < n) nd[mask | (1 << j)] = (nd[mask | (1 << j)] + v) % MOD;
                if (j + 1 < m && !(mask >> (j + 1) & 1))
                    nd[mask | (1 << (j + 1))] = (nd[mask | (1 << (j + 1))] + v) % MOD;
            }
            swap(dp, nd);
        }
    return dp[0];
}

int main() {
    int n, m;
    cin >> n >> m;
    cout << dominoTilings(n, m) << '\n';
}
```

6.4 Aliens trick — maximum total of exactly k disjoint subarrays

Input: n k, then the array. Output: the maximum total sum of exactly k disjoint non-empty subarrays (adjacent subarrays allowed).

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

// Penalised DP: each subarray costs lambda. Returns (best value, MIN #subarrays among
optimal).
// Values use __int128: |lambda| * #subarrays can reach n * sum|a_i| (e.g. 2e5 * 2e14).
typedef __int128 LL;
pair<LL, int> solvePenalized(const vector<ll>& a, ll lambda) {
    auto better = [](pair<LL, int> x, pair<LL, int> y) {
        return x.first != y.first ? x.first > y.first : x.second < y.second;
    };
    const LL NEG = -((LL)1 << 120);
    pair<LL, int> out = {0, 0}, in = {NEG, 0};              // not inside / inside a
subarray
    for (ll x : a) {
```

```
        pair<LL, int> from = better(out, in) ? out : in;     // a new subarray may start
right after one ends
        pair<LL, int> startNew = {from.first - lambda + x, from.second + 1};
        pair<LL, int> extend = {in.first + x, in.second};
        in = better(startNew, extend) ? startNew : extend;
        out = from;
    }
    return better(out, in) ? out : in;
}

ll exactlyK(const vector<ll>& a, int k) {                    // 1 <= k <= n
    ll B = 1;
    for (ll x : a) B += llabs(x);                            // every slope f(k)-f(k-1)
is in [-B, B]
    ll lo = -B, hi = B;
    while (lo < hi) {                                        // smallest lambda with
cnt_min <= k
        ll mid = lo + (hi - lo) / 2;
        if (solvePenalized(a, mid).second <= k) hi = mid; else lo = mid + 1;
    }
    return (ll)(solvePenalized(a, lo).first + (LL)lo * k);   // k, not the returned count
}

int main() {
    int n, k;
    cin >> n >> k;
    vector<ll> a(n);
    for (auto& x : a) cin >> x;
    cout << exactlyK(a, k) << '\n';
}
```

Overflow check: with $|a_i| \le 10^9$ and $n \le 2 \cdot 10^5$ we have $B \le 2 \cdot 10^{14}$, and the penalised value can reach $n \cdot B \approx 4 \cdot 10^{19}$ — beyond `long long`, hence `__int128` inside the DP. The final answer $f(k) \le \Sigma|a_i|$ fits in `long long`. Always redo this bound for your problem: the penalty term $\lambda \cdot \text{count}$ is the one that overflows.

6.5 Slope trick — CF 713C *Sonya and Problem Wihtout a Legend*

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct SlopeTrick {
    ll minf = 0, addL = 0, addR = 0;
    priority_queue<ll> L;                                    // left breakpoints (max-
heap)
    priority_queue<ll, vector<ll>, greater<ll>> R;           // right breakpoints (min-
heap)
    ll topL() const { return L.empty() ? LLONG_MIN / 4 : L.top() + addL; }
    ll topR() const { return R.empty() ? LLONG_MAX / 4 : R.top() + addR; }
    void pushL(ll a) { L.push(a - addL); }
    void pushR(ll a) { R.push(a - addR); }
```

```
    ll popL() { ll v = topL(); L.pop(); return v; }
    ll popR() { ll v = topR(); R.pop(); return v; }
    void addXminusA(ll a) { minf += max(0LL, topL() - a); pushL(a); pushR(popL()); }   // + max(0, x - a)
    void addAminusX(ll a) { minf += max(0LL, a - topR()); pushR(a); pushL(popR()); }   // + max(0, a - x)
    void addAbs(ll a) { addXminusA(a); addAminusX(a); }                                // + |x - a|
    void prefixMin() { while (!R.empty()) R.pop(); }        // f(x) = min_{y <= x} f(y)
    void suffixMin() { while (!L.empty()) L.pop(); }        // f(x) = min_{y >= x} f(y)
    void slideMin(ll a, ll b) { addL += a; addR += b; }     // f(x) = min_{x-b <= y <= x-a} f(y)
    ll eval(ll x) const {
        ll r = minf;
        auto l = L; auto rr = R;
        while (!l.empty()) { r += max(0LL, l.top() + addL - x); l.pop(); }
        while (!rr.empty()) { r += max(0LL, x - rr.top() - addR); rr.pop(); }
        return r;
    }
};

int main() {
    int n;
    cin >> n;
    SlopeTrick st;
    for (int i = 0; i < n; i++) {
        ll a;
        cin >> a;
        st.prefixMin();
        st.addAbs(a - i);       // strictly increasing a  <=>  non-decreasing a_i - i
    }
    cout << st.minf << '\n';
}
```

7. Common mistakes

1. **SOS loop order.** The bit loop must be outermost. Swapping it with the mask loop computes something else entirely and still passes tiny tests.
2. **Subset convolution without ranks** computes the OR convolution (overlapping pairs counted).
3. **Profile DP writing two bits without checking the second.** A horizontal domino needs bit j+1 to be free.
4. **Profile over the long side.** 2^{1000} states — always swap so the mask covers min(n, m).
5. **Aliens with inconsistent tie-breaking.** Taking the max count in one transition and the min in another breaks the monotonicity the binary search relies on.
6. **Aliens: using the count returned by the DP in the final formula.** Use k.
7. **Aliens: too narrow a λ range.** Slopes can be as large as the total of all values, not just the maximum element.

8. **Aliens on a non-concave function.** Passes samples, fails hidden tests. Check second differences by brute force first.
9. **Slope trick sentinels.** `LLONG_MIN` plus a lazy offset overflows; use ±LLONG_MAX/4 as in the reference code.
10. **Slope trick: forgetting the offset when pushing.** Always push `a - addL`, never `a`.

8. Practice problems

Problem	Technique
CF 165E — Compatible Numbers	SOS with witnesses
CF 449D — Jzzhu and Numbers	superset sums + inclusion–exclusion
CF 383E — Vowels	SOS over 2^{24} masks
CF 1208F — Bits And Pieces	SOS with top-2 indices
Library Checker — Subset Convolution	ranked zeta
CSES 2181 — Counting Tilings	broken profile
POJ 2411 — Mondriaan's Dream	broken profile
IOI 2016 — Aliens (oj.uz)	Aliens + CHT
CF 739E — Gosha is hunting	Aliens on one/two resources
CF 1279F — New Year and Handle Change	Aliens over #operations
CF 713C — Sonya and Problem Wihtout a Legend	slope trick
CF 13C — Sequence	slope trick (non-decreasing)
AtCoder ABC 217 H — Snuketoon	slope trick with sliding window
CF 1534G — A New Beginning	rotation + slope trick
APIO 2016 — Fireworks (oj.uz)	slope trick on trees, mergeable heaps

9. Further reading

- Codeforces blog by usaxena95, "SOS Dynamic Programming [Tutorial]" (2016).
- A. Björklund, T. Husfeldt, P. Kaski, M. Koivisto, "Fourier meets Möbius: fast subset convolution" (STOC 2007).
- cp-algorithms: "Dynamic Programming on Broken Profile" — https://cp-algorithms.com/
- Codeforces blog by Kaban-5, "The Trick From Aliens" and the IOI 2016 *Aliens* editorial; also "WQS binary search" write-ups (王钦石, 2012).
- Codeforces blog by maomao90, "Slope Trick Explained", and the AtCoder-community note "slope trick" by maspy.